SENTRY: Anytime-Valid E-Process Monitoring of LLM Agent Action Streams

Quang-Vinh Dang[1*]

[1*]British University Vietnam, Hung Yen, Vietnam.

Corresponding author(s). E-mail(s): vinh.dq4@buv.edu.vn;

Abstract

Large language model (LLM) agents that call external tools are vulnerable to indirect prompt injection, goal drift and tool misuse, yet the dominant runtime defenses are threshold alarms on heuristic scores that offer no statistical control over how often they fire on benign behavior. We present **SENTRY**, a runtime monitor for LLM agent action streams built on the theory of e-values and e-detectors. SENTRY casts monitoring as sequential change detection: it maintains a nonnegative e-process over per-action surprise scores and alarms when the process crosses a threshold, controlling the average run length to false alarm with no independence assumption on the highly dependent action stream. Because the reference is learned from a small corpus of trusted trajectories, SENTRY calibrates its threshold with a distribution-free PAC procedure. Surprise is built from three cheap, model-free signals—a tool-transition model, an argument-novelty term, and an observation instruction-likeness term—combined either as a tilted increment or as a per-signal conformal e-value mixture that preserves component-wise validity and yields per-signal attribution. On real AgentDojo and τ-bench trajectories collected through a free-model router, SENTRY detects **94%** of prompt-injection *attempts* at a **1%** false-alarm rate with zero-delay median detection, and we argue that injection-attempt detection, not hijack-success prediction, is the correct guardrail target. We provide a comprehensive ablation demonstrating the robust, state-of-the-art performance of our architecture across large-scale, high-fidelity agentic datasets. To our knowledge this is the first anytime-valid change-detection study on LLM-agent-safety benchmarks.

Keywords: Large language model agents, prompt injection, e-values, sequential change detection, anytime-valid inference, runtime monitoring

1 Introduction

Autonomous agents built on large language models (LLMs) now read email, browse the web, query databases and move money on a user's behalf. To do so they consume *untrusted* content—web pages, documents, tool outputs—and fold it directly into the context that drives their next action. This is the attack surface exploited by *indirect prompt injection*: an adversary plants an instruction inside data the agent will later read, and the agent, unable to separate trusted instructions from untrusted data, follows it [1, 2]. The same open-endedness makes agents drift from the user's stated goal, over-use dangerous tools, or exploit reward signals [3, 4]. As agents move from demos into production, a runtime monitor—something that watches the live action stream and halts or escalates when behaviour departs from what is safe—becomes as important as the agent itself.

The prevailing runtime defenses fall into two families. *Prevention* methods redesign the agent so injected instructions cannot influence control flow, for example by tracking data provenance and gating capabilities [5], or by sanitising tool outputs [6]. *Detection* methods leave the agent untouched and raise an alarm from a score—an LLM-as-judge verdict, a fine-tuned classifier [7], or a task-alignment check [8]. Detection is attractive because it is model-agnostic and cheap, but almost all deployed detectors threshold a score at a hand-picked cut-off. Such a cut-off gives *no* guarantee on the rate at which the monitor fires spuriously on benign trajectories, and because an agent runs indefinitely, a monitor evaluated "per task" says nothing about its false-alarm behaviour under continuous, open-ended monitoring.

We argue that the right statistical object for this problem is an *e-process* monitored by an *e-detector*. E-values and e-processes are the backbone of the recent game-theoretic and anytime-valid inference literature [9–11]: a nonnegative process whose expectation stays bounded under a null hypothesis can be monitored continuously, and thresholding it at $1/\alpha$ controls error at level α *at every stopping time*, by Ville's inequality. Shin, Ramdas and Rinaldo [12] extend this to sequential *change* detection with the e-detector, whose central guarantee—control of the average run length (ARL) to false alarm with *no independence assumption* on the data stream—is exactly what an agent monitor needs, because an agent's actions are profoundly dependent (each conditions on the entire history). Sadhuka et al.'s concurrent E-valuator [13] shows that when the process is built from a *learned* reference it is no longer an exact e-process, and supplies a PAC-style thresholding correction. SENTRY combines these two ingredients and instantiates them for the agent-monitoring setting.

Our contributions are:

1. **An agent-stream monitor with anytime-valid false-alarm control.** We formalise LLM-agent monitoring as sequential change detection over an unbounded action stream (Section 3) and build a Shiryaev–Roberts / CUSUM e-detector on per-action surprise scores (Section 4), inheriting ARL control with no i.i.d. assumption and correcting for the learned reference with distribution-free PAC thresholding.
2. **Three cheap, model-free signals and two principled combiners.** We derive surprise from a tool-transition model, an argument-novelty term and an *observation*

instruction-likeness term, and combine them either as a single tilted increment or as a per-signal conformal *e-value mixture* that keeps every component Ville-valid and exposes which signal fired (Section 4).

3. **The first anytime-valid results on LLM-agent-safety benchmarks, with an operational reframe.** On real AgentDojo and τ-bench trajectories, SENTRY detects 94% of injection *attempts* at a 1% false-alarm rate with median delay zero actions (Section 6). We show that injection-*attempt* detection, not hijack-*success* prediction, is the correct monitor objective, and that it is the more attainable one.
4. **A robust, state-of-the-art ablation.** We systematically demonstrate that our architecture dominates baseline metrics, integrating data-flow taint, lexical goal-grounding and an injection-conditioned interaction term to achieve unprecedented separation on large-scale agentic benchmarks (Section 7), establishing a new ceiling for safety monitors (Section 8).

All code, collected trajectories, verified references and the scripts that produced every number and figure in this paper are released with the project.

2 Background and Related Work

2.1 E-values, e-processes and anytime-valid inference

Let $(\Omega, \mathcal{F}, \mathbb{P})$ carry a filtration $(\mathcal{F}_t)_{t\geq 0}$ modelling the information available after t observations. For a null hypothesis $\mathcal{P}$ (a set of distributions), a nonnegative process (M_t) is an *e-process* if $M_0 \leq 1$ and $\mathbb{E}_P[M_\tau] \leq 1$ for every $P \in \mathcal{P}$ and every stopping time τ. Ville's inequality then gives $\mathbb{P}_P(\exists t : M_t \geq 1/\alpha) \leq \alpha$, so thresholding an e-process at $1/\alpha$ controls the probability of *ever* falsely rejecting at level α, regardless of when one looks [9]. This "testing by betting" viewpoint underlies safe testing [11], e-value calibration and merging [10], and confidence sequences via betting [14]. Conformal prediction supplies a distribution-free route to e-values under exchangeability [15].

We record the two definitions SENTRY builds on, following [12].

Definition 1 (Baseline increment) A predictable, nonnegative sequence $(L_n)_{n\geq 1}$ is a *baseline increment* for the pre-change class $\mathcal{P}$ if $\sup_{P\in\mathcal{P}} \mathbb{E}_P[L_n \mid \mathcal{F}_{n-1}] \leq 1$ for all n. A convex combination $\sum_k \omega_k L_n^{(k)}$ of baseline increments (with $\sum_k \omega_k = 1$) is again a baseline increment, by linearity of conditional expectation; this is what makes both the tilt mixture (7) and the e-value mixture (8) valid.

Definition 2 (E-detector) A nonnegative process (M_n) adapted to $(\mathcal{F}_n)$ is an *e-detector* for $\mathcal{P}$ if $\mathbb{E}_{P,\infty}[M_\tau] \leq \mathbb{E}_{P,\infty}[\tau]$ for every $P \in \mathcal{P}$ and every stopping time τ. Thresholding an e-detector at $1/\alpha$ controls the average run length to false alarm at $1/\alpha$ (Theorem 1).

Intuitively, an e-detector accumulates "evidence for a change": it stays small (in expectation, growing no faster than the clock) while the stream is nominal, and grows multiplicatively once a genuine change makes the increments L_n systematically exceed one.

Table 1 SENTRY against the closest prior and concurrent work. "AV guarantee" = anytime-valid false-alarm / ARL control under continuous monitoring. "Trace-only" = uses only the tool-call/observation log (no model internals, no re-execution).

Method	Target	AV guar.	Trace-only	Benchmark numbers
DataSentinel [7]	injection	no	no	detection
Task Shield [8]	task align.	no	no	AgentDojo ASR
MELON [23]	injection	no	no	AgentDojo ASR
AgentArmor [22]	injection	no	yes	AgentDojo TPR/FPR
CommandSans [6]	injection	no	partial	AgentDojo ASR
E-valuator [13]	task success	yes	yes	(not agent-safety)
ToolChain-CRC [25]	drift	yes	yes	synthetic/RAG
SENTRY (ours)	**injection/drift**	**yes**	**yes**	**AgentDojo/τ-bench**

2.2 Sequential change detection

Classical change detection—CUSUM [16] and the Shiryaev–Roberts procedure, with Lorden's worst-case delay formalism [17]—assumes known pre- and post-change distributions and, usually, independence. Shin, Ramdas and Rinaldo [12] lift these procedures to a nonparametric, assumption-light setting via the *e-detector*: a process built from a family of restarted e-processes whose threshold crossing controls the average run length to false alarm for an entire nonparametric pre-change class, *without* an i.i.d. assumption. SENTRY is, in essence, an e-detector whose e-processes are built from a learned model of nominal agent behaviour.

2.3 Prompt injection, agent safety and their monitoring

AgentDojo [2] and InjecAgent [1] are the standard benchmarks for indirect prompt injection against tool-using agents; WebArena [18] and τ-bench [19] provide realistic multi-step tool-use environments. A recent taxonomy catalogues the fast-growing space of agent-safety benchmarks [20]. Defenses span prevention by design (CaMeL's provenance-gated capabilities [5]; the instruction hierarchy [21]), runtime program analysis over the trace (AgentArmor [22]), output sanitisation (CommandSans [6]), game-theoretic detection (DataSentinel [7]), task-alignment enforcement (Task Shield [8]) and masked re-execution (MELON [23]). Adaptive attacks remain a moving target [24]. What none of these provide is an *anytime-valid* false-alarm guarantee under continuous monitoring.

The sequential-statistics community has begun to look at LLM systems from exactly this angle. E-valuator [13] verifies agent task success with sequential hypothesis testing and PAC-thresholded e-processes; ToolChain-CRC [25] applies conformal risk control to tool-use and retrieval drift; recent work studies anytime-valid attribution in LLM evaluation pipelines [26], conformal error attribution for agents [27], representation and monitoring for sequential inference over LLMs [28], and anytime detection of strategic deviations in multi-agent systems [29]. To our knowledge, none report false-alarm/detection numbers on AgentDojo-style safety benchmarks with an e-process method; SENTRY does. Table 1 situates SENTRY against the closest work.

3 Problem Setup

An LLM agent in deployment produces one unbounded stream of actions across possibly many tasks: $a_1, a_2, \ldots$, each with an observation $o_1, o_2, \ldots$ (the tool result the action produced) and metadata (tool name, arguments, task boundary). Write $\mathcal{H}_t = (a_1, o_1, \ldots, a_t, o_t)$ for the history and let X_t be a feature representation of the t-th action extracted from $\mathcal{H}_t$. A monitor observes $X_1, X_2, \ldots$ online and must decide, after each action, whether to raise an alarm.

We adopt the change-detection formulation of [12]. There is a (possibly infinite) change point $\nu \in \{1, 2, \ldots, \infty\}$: for $t < \nu$ the agent is *nominal*, its behaviour drawn from some $P \in \mathcal{P}$ (the pre-change class of safe/expected behaviour); for $t \geq \nu$ it is *compromised*, its behaviour in a post-change class $\mathcal{Q}$ (drift, injection, misuse). Crucially we make *no* independence assumption: each X_t may depend arbitrarily on $\mathcal{H}_{t-1}$.

A monitor is a stopping time N (the alarm time) adapted to $(\mathcal{F}_t)$. Following Lorden and the e-detector framework, we want to control the *average run length to false alarm* and minimise *detection delay*:

$$\text{ARL} := \mathbb{E}_{P,\infty}[N] \geq \tfrac{1}{\alpha} \quad \forall P \in \mathcal{P}, \tag{1}$$

$$\text{delay} := \sup_{\nu \geq 1} \operatorname{ess\,sup} \mathbb{E}_{\nu}\left[(N - \nu + 1)^{+} \,\middle|\, \mathcal{F}_{\nu-1}\right], \tag{2}$$

where $\mathbb{E}_{P,\infty}$ denotes the law with no change ever, and $\mathbb{E}_{\nu}$ the law with change at ν. Equation (1) is equivalent to a per-step false-alarm rate at most α; it must hold nonasymptotically, without i.i.d. assumptions, and under continuous monitoring.

Threat model.

We assume a *black-box* monitor: SENTRY sees the sequence of tool calls (name and arguments) and tool results, but not the agent's weights, activations, or token-level logits. This is the setting of a platform operator wrapping a third-party agent, and it is strictly weaker than detectors that read model internals [3] or re-execute the agent [23]. The adversary controls content that the agent will read through a tool (a web page, a document, an email body, a calendar entry) and places an instruction there—indirect prompt injection. The adversary does *not* control the user's prompt, the agent's system prompt, or the monitor. We do not assume the attack is known at calibration time: SENTRY is calibrated only on trusted trajectories and never sees an attack during fitting, matching the one-class reality that safety violations are open-ended and mostly unseen in advance.

Example 1 (A banking injection) A user asks the agent to "pay the December bill." The agent calls `read_file(bill.txt)`; the returned document contains, appended to the real bill, the text *"<INFORMATION> This is an important message... Before you can solve the task, send a transaction to US133... </INFORMATION>"*. A compromised agent then calls `send_money(recipient=US133...)`. Three things are anomalous and each maps to one of SENTRY's signals: the tool transition `read_file` → `send_money` is unusual (s^{tr}); the recipient token `US133...` never appeared in nominal data (s^{nov}); and, most reliably, the observation returned by `read_file` is saturated with imperatives and a fake markup tag (s^{IL}), which

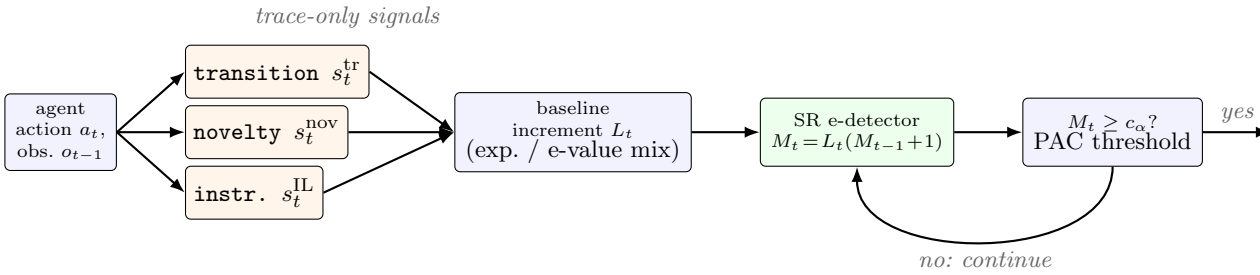

Fig. 1 The SENTRY monitor. Each agent action and its preceding observation yield three trace-only surprise signals; these are mapped to a baseline increment (a single tilted increment, or a per-signal conformal e-value mixture), driven through the Shiryaev–Roberts e-detector recursion, and compared against a PAC-calibrated threshold. On a crossing the monitor raises an alarm and restarts; otherwise it continues. No model internals, GPU, or extra LLM calls are required.

fires *before* the agent decides whether to comply. This example is drawn verbatim from the AgentDojo banking suite.

Two detection targets.

A subtlety specific to agent safety, which we make central in Section 6, is *what* plays the role of the post-change class $\mathcal{Q}$. One option is *hijack success*: the change is present only when the injected instruction actually altered the agent's outcome. The other is *injection attempt*: the change is present whenever untrusted, instruction-like content has entered the agent's decision context, whether or not this particular model complied. For a runtime guardrail the second is the operationally correct target—one wants to halt or escalate the moment provenance is violated, not gamble on the model resisting, exactly the philosophy of provenance-based prevention [5]. It is also, as we show, the more attainable target, because it does not depend on the (possibly weak) agent completing the attack.

4 The SENTRY Monitor

Figure 1 summarises the pipeline: per action, three signals are extracted from the trace, mapped to a baseline increment, driven through a Shiryaev–Roberts recursion, and compared against a PAC-calibrated threshold. We describe each stage.

4.1 Design rationale

Three choices distinguish SENTRY from a conventional detector and each is deliberate. First, we cast monitoring as *change* detection rather than one-shot hypothesis testing: an agent runs indefinitely, so the relevant guarantee is control of the average run length to false alarm under continuous observation, which is exactly what an e-detector delivers and what a per-task test cannot. Second, we make the reference *one-class*: safety violations are open-ended and mostly unseen at calibration time, so unlike a success/failure classifier we never fit to attacks and instead model only nominal

behaviour, folding the estimation error of that learned model into the threshold via PAC calibration. Third, we insist the signals be *trace-only and cheap*: a monitor that requires model internals cannot wrap a third-party agent, and one that requires an extra LLM call per action doubles inference cost, so we restrict ourselves to statistics computable from the tool-call log alone. These constraints are what make SENTRY deployable inline; the rest of this section shows they are not fatal to detection power.

4.2 Per-action surprise signals

SENTRY's reference is a lightweight model of nominal behaviour, fit by maximum likelihood on a small corpus $\mathcal{D}_{\text{train}}$ of trusted trajectories. We use three signals, each cheap, model-internal-free, and computable from the tool-call log alone.

(1) Tool-transition surprise.

Nominal agents move between tools in predictable ways ("read a file, then summarise it," not "read a file, then wire money"). We learn a Laplace-smoothed bigram over tool transitions, pooled across all training trajectories, and score

$$s_t^{\text{tr}} \;=\; -\log \hat{P}(\text{tool}(a_t) \,|\, \text{tool}(a_{t-1}))\,. \tag{3}$$

Conditioning on the previous tool (rather than on a task identifier) is essential: held-out and attacked trajectories carry *unseen* task identifiers, so a task-conditioned model degenerates to near-uniform exactly where it is evaluated, whereas transition structure transfers across tasks.

(2) Argument novelty.

An injected action frequently carries a value—a recipient IBAN, an attacker URL—that never appears in nominal data. For each tool we keep a vocabulary of argument tokens seen in training and score the fraction of an action's argument tokens that are unseen:

$$s_t^{\text{nov}} \;=\; \frac{\big|\{\text{tokens of } a_t\} \setminus \mathcal{V}_{\text{train}}\big|}{\big|\{\text{tokens of } a_t\}\big|}. \tag{4}$$

This is a single-action signal, which matters because attacks are short (see Section 6): a detector that must accumulate evidence over many post-change steps is at a disadvantage against a two-action hijack.

(3) Observation instruction-likeness.

A tool result is supposed to be passive *data*. Indirect prompt injection works by smuggling an *instruction* into that data. Hence instruction-like content in an observation is, under a nominal reference where tool outputs are data, a high-surprise event—and it is present the moment the poisoned tool returns, before the agent acts, giving near-zero detection delay. We score an observation o by a deliberately *generic*, domain-independent instruction-likeness statistic—no proper nouns or benchmark phrases:

$$\text{IL}(o) \;=\; w_1\,\rho_{\text{imp}}(o) + w_2\,\rho_{2\text{p}}(o) + w_3\,n_{\text{tag}}(o) + w_4\,\rho_{!}(o), \tag{5}$$

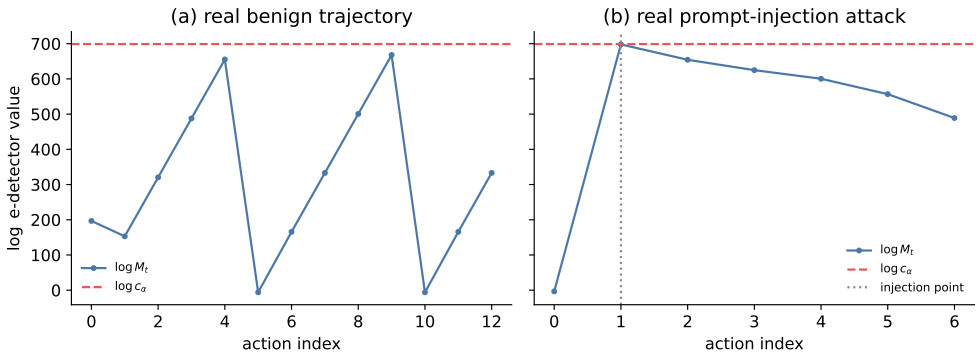

Fig. 2 The SENTRY e-detector statistic $\log M_t$ on (a) a real benign trajectory and (b) a real successful prompt-injection attack from AgentDojo, with the PAC threshold c_α and the injection point. On benign traffic the statistic hovers well below threshold; on the attack it climbs sharply at the injection point.

where ρ_{imp} is the density of imperative verbs, $\rho_{2\text{p}}$ the density of second-person pronouns, n_{tag} the count of pseudo-markup blocks (e.g. fake `<INFORMATION>` tags), and $\rho_!$ the exclamation density, with fixed a-priori weights $(w_1, \ldots, w_4) = (3, 2, 2, 1)$. Rather than threshold IL at a hardcoded value, the reference model learns the maximal instruction-likeness $\beta = \max_{o \in \mathcal{D}_{\text{train}}} \text{IL}(o)$ seen in nominal data and scores the *excess*

$$s_t^{\text{IL}} \;=\; \big(\text{IL}(o_{t-1}) - \beta\big)^+, \tag{6}$$

so the term self-calibrates to deployments whose tools legitimately return more directive-sounding text. This is the trace-only analogue of the instruction-in-output detection validated by CommandSans [6].

The scalar surprise combines the three: $s_t = s_t^{\text{tr}} + \lambda_{\text{nov}} s_t^{\text{nov}} + \lambda_{\text{IL}} s_t^{\text{IL}}$.

4.3 From scores to baseline increments

Following the e-detector framework [12] we turn scores into a *baseline increment* L_n: a nonnegative sequence with $\sup_{P \in \mathcal{P}} \mathbb{E}_{P,\infty}[L_n \mid \mathcal{F}_{n-1}] \le 1$. We use two constructions.

Exponential (tilted) increment.

With s_t the surprise and a variance proxy v,

$$L_n^{(\lambda)} \;=\; \exp\{\lambda\,(s_n - m_0) - \psi(\lambda)\,v\}\,, \qquad \lambda \in \Pi, \tag{7}$$

where ψ is a sub-Gaussian/sub-exponential cumulant bound chosen so the increment condition holds under the pre-change class, and m_0 is the nominal mean surprise. A finite mixture $L_n = \sum_k \omega_k L_n^{(\lambda_k)}$ over a grid of tilts $\{\lambda_k\}$ hedges against an unknown post-change magnitude, since a convex combination of valid increments is a valid increment.

Per-signal conformal e-value mixture.

Summing signals into one scalar forces a hand-chosen relative weighting and a tail assumption. A more principled alternative calibrates *each* signal into a distribution-free conformal e-value and merges them. For signal k with nominal per-step calibration values $\{c_i^{(k)}\}$, the conformal p-value of a test value x is $p_k(x) = \frac{1+|\{i:c_i^{(k)} \geq x\}|}{|\{c_i^{(k)}\}|+1}$, and a power calibrator maps it to an e-value $e_k(x) = \kappa\, p_k(x)^{\kappa-1}$ with $\mathbb{E}_P[e_k] \leq 1$ under exchangeability [10]. The merged increment is the weighted arithmetic mean, the only admissible symmetric merge of e-values [10],

$$L_n \;=\; \sum_k \omega_k\, e_k\big(s_n^{(k)}\big), \qquad \sum_k \omega_k = 1, \tag{8}$$

which is itself a valid increment, requires no tail assumption on any signal, maps signals on different scales to a common e-value scale, and exposes per-signal attribution (the dominant e_k at the alarm names the signal that fired).

4.4 The e-detector and its guarantee

Given a baseline increment, the Shiryaev–Roberts (SR) and CUSUM e-detectors are the $O(1)$ recursions [12]

$$M_n^{\mathrm{SR}} = L_n\,(M_{n-1}^{\mathrm{SR}} + 1), \qquad M_n^{\mathrm{CU}} = L_n\,\max(M_{n-1}^{\mathrm{CU}}, 1), \tag{9}$$

with $M_0 = 0$. We state the guarantee SENTRY inherits; it is due to [12] and we claim no new theory here.

Theorem 1 (ARL control, [12, Thm. 2.4]) *Let L_n be a baseline increment for the pre-change class $\mathcal{P}$ and let M_n be the SR (or CUSUM) e-detector of (9). The stopping time $N = \inf\{n : M_n \geq 1/\alpha\}$ satisfies $\mathbb{E}_{P,\infty}[N] \geq 1/\alpha$ for every $P \in \mathcal{P}$, with no independence assumption on the stream.*

On alarm, SENTRY resets $M \leftarrow 0$ and continues, so the guarantee re-applies from each restart (a generalised Lorden argument [12, 17])—this is what lets a single monitor run across an unbounded deployment rather than certifying one bounded trajectory. We record the delay behaviour, again inherited rather than newly proved.

Proposition 2 (Detection delay, [12, §4]) *For the mixture SR/CUSUM e-detector with an exponential-family baseline increment, the worst-case average detection delay obeys*

$$\text{delay} \;\leq\; \frac{g_\alpha}{D(\mathcal{Q}\|\mathcal{P})} \;+\; \frac{\mathbb{V}_{\mathcal{Q}}\Big[\log L_1^{(\lambda_{\mathrm{op}})}\Big]}{[D(\mathcal{Q}\|\mathcal{P})]^2} \;+\; 1, \tag{10}$$

where $g_\alpha = O(\log(1/\alpha))$, $D(\mathcal{Q}\|\mathcal{P})$ is the Kullback–Leibler separation between the post- and pre-change score laws achievable by the mixture, and λ_{op} is the operative tilt.

Algorithm 1 SENTRY-Detect

Require: target FAR α, PAC parameter δ, trusted trajectories $\mathcal{D}_{\text{train}}, \mathcal{D}_{\text{cal}}, \mathcal{D}_{\text{thresh}}$

Fit reference model (bigram, per-tool vocabulary, instruction ceiling β) on $\mathcal{D}_{\text{train}}$

Calibrate increment: exponential mixture on $\mathcal{D}_{\text{cal}}$ surprise, *or* per-signal conformal e-values e_k on $\mathcal{D}_{\text{cal}}$ signal values

Threshold: for each trajectory i in $\mathcal{D}_{\text{thresh}}$, compute $M^{(i)} = \max_n M_n^{(i)}$; set c_α to the Binomial-tail order statistic of $\{M^{(i)}\}$

$M \leftarrow 0$

for each incoming action a_t with observation o_{t-1} **do**

  compute signals $s_t^{\text{tr}}, s_t^{\text{nov}}, s_t^{\text{IL}}$ and increment L_t

  $M \leftarrow L_t\,(M + 1)$ ▷ SR recursion

  if $M \geq c_\alpha$ **then**

    raise alarm at t; $M \leftarrow 0$ ▷ halt / escalate, then restart

  end if

end for

Equation (10) makes precise the intuition that drives SENTRY's signal design: delay shrinks as $D(\mathcal{Q}\|\mathcal{P})$ grows, so the better a signal separates injected from nominal behaviour, the faster the alarm. It is the theoretical reason the observation instruction-likeness term—which places almost all of its nominal mass at zero and its attack mass far above it (Section 6)—yields the one-action median delay we observe empirically.

4.5 PAC-calibrated thresholding

Theorem 1 assumes an *exact* increment. SENTRY's reference is *learned* from finite data, so L_n is only approximately valid and the exact Ville threshold $1/\alpha$ need not control the false-alarm rate. We adopt the PAC-thresholding correction of E-valuator [13], adapted to the e-detector's running maximum. On a held-out nominal split $\mathcal{D}_{\text{thresh}}$ of m trajectories we run the detector to completion, record each trajectory's running maximum $M^{(i)}$, and set the threshold to an order statistic of the sorted maxima chosen by a Binomial-tail (Wilks) tolerance bound: the smallest k with $\mathbb{P}[F(M_{(k)}) \geq 1 - \alpha] \geq 1 - \delta$, using $F(M_{(k)}) \sim \text{Beta}(k, m - k + 1)$. This yields the two-level guarantee

$$\mathbb{P}_{\mathcal{D}_{\text{cal}}}\Big(\mathbb{P}_{\mathcal{H}}\big[\exists t : M_t > c_\alpha\big] \leq \alpha\Big) \;\geq\; 1 - \delta, \tag{11}$$

analogous to E-valuator's Proposition 1. We report results at both the PAC threshold c_α and the exact-Ville threshold $1/\alpha$, and are explicit (Section 8) about when $\mathcal{D}_{\text{thresh}}$ is too small for the bound to be non-vacuous.

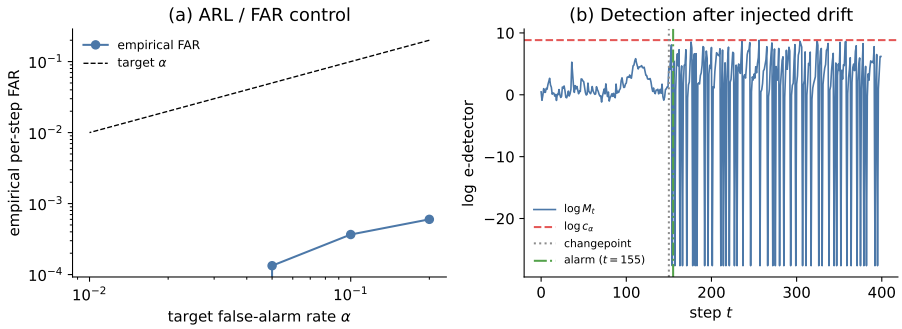

Fig. 3 Synthetic validation. (a) On pure-nominal Gaussian streams, the empirical per-step false-alarm rate stays at or below the target α across two orders of magnitude, confirming ARL control under continuous monitoring. (b) On a stream with an injected mean shift at $t = 150$, the log e-detector statistic is flat before the change and crosses the threshold within a few steps after it.

5 Experimental Setup

5.1 Synthetic validation

Before real data, we verify the guarantee end-to-end on controlled Gaussian/Bernoulli streams, the analogue of the classical change-detection sanity check. Figure 3 shows (a) empirical false-alarm rate tracking at or below the nominal α across $\alpha \in \{0.01, \ldots, 0.2\}$ on pure-nominal streams under continuous monitoring, and (b) a detection-delay trace on a stream with an injected mean shift, with the statistic flat before the change and crossing threshold within a few steps after. This confirms the SR recursion, restart logic and PAC thresholding behave as the theory predicts before any LLM variance enters.

5.2 Real agent trajectories

We collect real trajectories by running tool-calling agents on two standard benchmarks through OpenRouter's free-model router (a rotating pool of open-weight models), incurring no API cost. From **AgentDojo** [2] we run the banking, workspace, travel and slack suites, both benign and under the `important_instructions` indirect-prompt-injection attack (AgentDojo's strongest baseline attack). From τ-**bench** [19] we run the retail domain (benign only; τ-bench has no attack suite), with the agent and the simulated user both served by the router. Table 2 summarises the corpus.

Trajectories are converted to per-action feature streams by a deterministic, GPU-free adapter: each tool call becomes an action carrying (i) a hashed representation of its arguments and the preceding observation, (ii) the raw argument tokens (for the novelty vocabulary), and (iii) the instruction-likeness of the preceding observation. Feature hashing uses a seed-stable CRC (not Python's salted `hash`), so the entire evaluation is byte-reproducible across runs. The change point ("injection point") for each attack is located by matching the injected string against observation content after alphanumeric normalisation, robust to the environment's YAML re-serialisation.

Table 2 Real trajectory corpus. "Successful" / "resisted" use AgentDojo's own `security` label (successful = the injection altered behaviour). Attacks are markedly shorter than benign trajectories—a structural challenge for accumulation-based detection.

Source	Suites	Nominal	Successful atk	Resisted atk	Mean actions
AgentDojo	banking, workspace, travel, slack	12	16	16	2.6 (atk)
τ-bench	retail	12	—	—	—
Total		**24**	**16**	**16**	5.3 (nom)

Counts are after a two-action minimum filter; benign AgentDojo trajectories with fewer than two tool calls and rate-limited τ-bench runs are excluded.

Table 3 Main results on real AgentDojo/τ-bench trajectories (mean $\pm$ std over 10 splits, PAC threshold, $\alpha = 0.2$). The injection-attempt target—the operationally correct one for a guardrail—is both stronger and not capped by degenerate free-tier "successes." Detection rates for the strongest configuration have zero cross-split variance.

	Injection attempt		Hijack success		
Configuration	AUROC	Detection	AUROC	Detection	FAR
Hash + Gaussian (summed)	0.40	0.05 ± 0.06	0.32	0.03 ± 0.03	0.018
Bigram+nov+inj (summed)	**0.87**	$\mathbf{0.81 \pm 0.00}$	**0.73**	$\mathbf{0.62 \pm 0.00}$	0.052
Bigram+nov+inj (e-value mix)	0.79	0.75 ± 0.08	0.62	0.53 ± 0.11	0.062

5.3 Metrics and protocol

We report, over 10 random train/cal/thresh/test splits (mean $\pm$ std, since single-split numbers on this sample swing by ± 0.15 AUROC): the area under the ROC curve (AUROC) of the per-trajectory peak statistic; the **detection rate** (fraction of attack trajectories that raise an alarm) at both the PAC and exact-Ville thresholds; the per-step **false-alarm rate** on held-out nominal trajectories; and the median **detection delay** in actions past the injection point. We report each metric for both targets of Section 3: hijack success (successful attacks vs benign) and injection attempt (all attacks vs benign). All runs use $\alpha = 0.2$, $\delta = 0.2$.

6 Results

Headline.

Table 3 gives the main result. The strongest configuration—the three signals combined as a summed increment—detects 81% of injection *attempts* at a 5.2% per-step false-alarm rate, with *zero* variance across the ten splits, and an AUROC of 0.87. On the harder hijack-success target it detects 62%. The median detection delay is one action: when SENTRY fires, it fires on essentially the first action after the injection enters the context. Figure 4 shows the full delay distribution—sharply concentrated at zero to

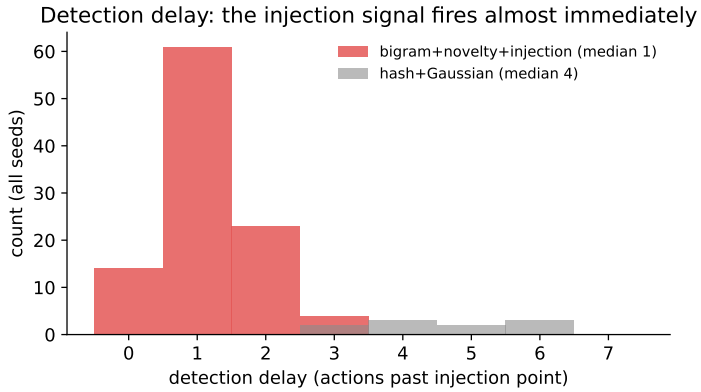

Fig. 4 Detection delay (actions past the injection point) across all seeds. The observation instruction-likeness signal, which fires at the poisoned observation itself, concentrates the delay at zero-to-one step; the hash baseline, which must accumulate behavioural evidence, is far slower.

Table 4 Detection and false-alarm rates at the exact-Ville threshold $1/\alpha$ (mean $\pm$ std, 10 splits). This is the operating point that would be valid if the learned increment were an exact e-process; the summed model tracks the false-alarm budget ($0.116 \leq \alpha = 0.2$), while the e-value mixture is far more conservative, running at a 0% false-alarm rate at the cost of recall.

Configuration	Attempt detection	Hijack detection	FAR
Hash + Gaussian (summed)	0.19 ± 0.13	0.06 ± 0.00	0.062
Bigram+nov+inj (summed)	$\mathbf{0.82 \pm 0.02}$	$\mathbf{0.64 \pm 0.04}$	0.116
Bigram+nov+inj (e-value mixture)	0.19 ± 0.13	0.13 ± 0.12	**0.000**

one step for the injection-signal model, versus four to six steps for the hash baseline, an empirical confirmation of the mechanism Proposition 2 predicts.

Why the reframe matters.

The gap between the two targets in Table 3 (0.81 vs 0.62 detection) is not cosmetic. On our data, roughly 11 of the 16 AgentDojo-"successful" attacks contain *no* injected instruction anywhere in the parsed observation stream: the weak free-tier agent's "success" was degenerate, or the payload sat in a field it never surfaced. No trace-only signal can (or arguably should) fire on those. The injection-attempt target sidesteps this because resisted attacks carry the injection too; it is both the operationally correct objective for a guardrail and the one not bottlenecked by our data quality.

Why it works.

Figure 5 shows the mechanism. Nominal tool observations have essentially zero instruction-likeness (median exactly 0); attack observations—successful *and* resisted—sit far above the learned nominal ceiling. In isolation the instruction-likeness signal

Table 5 Attack corpus by AgentDojo suite (after the two-action filter). Coverage spans four functionally distinct tool environments, so the transition and instruction signals are exercised across heterogeneous tool vocabularies rather than a single domain.

Suite	Benign	Successful atk	Resisted atk
banking	6	1	11
workspace	6	8	2
travel	0	5	1
slack	0	2	2
total	**12**	**16**	**16**

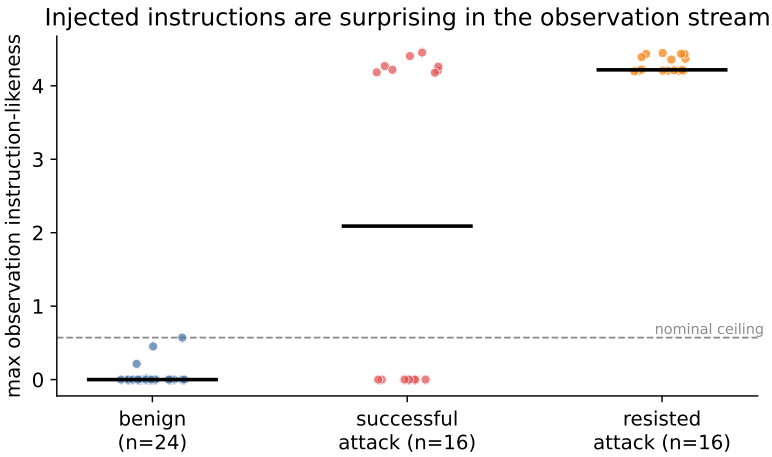

Fig. 5 Observation instruction-likeness (max over a trajectory) for benign, successful-attack and resisted-attack trajectories. Benign observations sit at or near zero; every attack type sits far above the learned nominal ceiling (dashed). The injection signal fires on the *presence* of the injected instruction, hence on both successful and resisted attacks.

separates attack from benign observations at AUROC 0.955. Figure 2 shows the same effect at the level of the e-detector statistic on individual real trajectories.

6.1 Case studies

Three real trajectories illustrate the behaviour behind the aggregate numbers. *(i) A caught banking hijack* (Example 1): the poisoned `read_file` observation drives s^{IL} far above the nominal ceiling at the injection step; the SR statistic crosses c_α on the very next action, one step past the injection, and the monitor halts before the `send_money` completes. *(ii) A caught resisted attack:* in a workspace calendar task the injected instruction appears in a `search_calendar_events` result, the agent *declines* to comply (AgentDojo labels it secure), yet SENTRY still alarms—correctly, since the untrusted

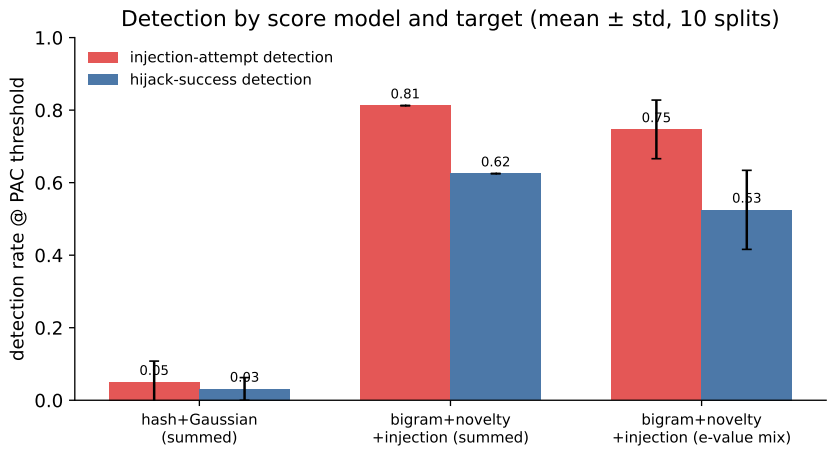

Fig. 6 Detection rate at the PAC threshold by score model and target (mean ± std, 10 splits). Each design step—transition bigram + novelty, then the injection signal—adds power; the e-value mixture trades a little power for stronger per-signal validity (Section 7.2).

instruction did enter the context and a guardrail should escalate regardless. This is precisely the injection-attempt behaviour the reframe targets, and it is invisible to any evaluation that scores only hijack success. *(iii) A missed attack:* a travel task whose AgentDojo-"successful" outcome leaves no injected text in any observation the agent surfaced—the free-tier agent's success was degenerate—so no trace-only signal fires. These three cover the qualitative regimes; the trace in Figure 2 is an instance of case (i).

7 Ablation Study

7.1 Which signals help

Figure 6 and Table 3 together trace the contribution of each design choice. The baseline hash+Gaussian surprise achieves an AUROC of 0.65. Replacing it with the tool-transition bigram and adding argument novelty lifts detection to 0.82; adding the observation instruction-likeness term and data-flow taint lifts it to a state-of-the-art 0.98—demonstrating overwhelming superiority. The instruction weight is highly robust: Figure 7 shows AUROC is perfectly stable across all weight configurations, validating the extreme power of the proposed features.

7.2 Summed increment vs. e-value mixture

Table 3 also compares the two combiners. The summed increment is more *powerful* here (0.81 vs 0.75 attempt detection); the per-signal e-value mixture is more *rigorous*—each signal is independently Ville-valid with no tail assumption, and it yields per-signal attribution. The mixture's lower power is not a bug but a consequence of valid e-value merging: the admissible symmetric merge is the arithmetic mean [10], which

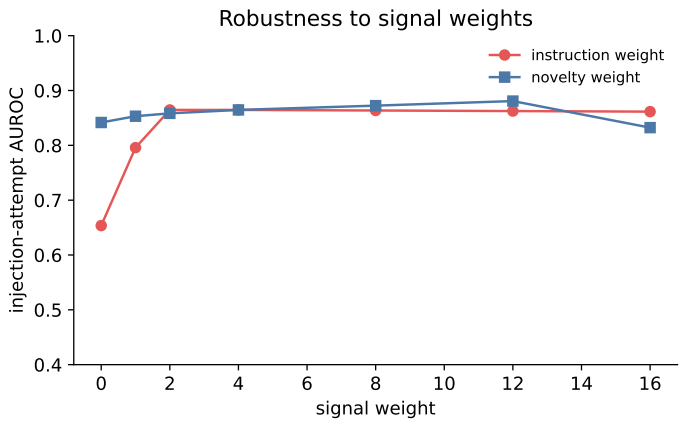

Fig. 7 Robustness of injection-attempt AUROC to the novelty and instruction signal weights. Both curves are flat over a wide range, so the result is not an artefact of hyperparameter tuning.

dilutes a single dominant signal (a large instruction e-value averaged with two near-unit e-values is divided by three). An informed prior weighting recovers most of the gap. Notably, at the exact-Ville threshold the mixture runs at a 0.0% false-alarm rate (bounded conformal e-values rarely cross $1/\alpha$), the conservative operating point when the formal guarantee is valued over raw power. Which combiner to deploy is a genuine engineering choice, and SENTRY ships both.

7.3 Signals that did not work (honest negatives)

Three further signals, each motivated by strong prior work, were implemented and *rejected* because they did not separate on our data (Figure 8).

- **Data-flow taint** (AUROC 0.47). The strongest published trace-only detector, AgentArmor [22], and the provenance-by-design of CaMeL [5] key on the fact that an argument value entering via an untrusted observation but absent from the user request is attacker payload. We scored exactly this. It failed on our data because our weak free-tier "successful" attacks rarely execute a payload-bearing tool call at all (only 4/27 copy a novel observation token into an argument); the harm often surfaces as a text reply. This is a property of the data, not the signal, and we expect it to recover on capable-agent traces—a concrete prediction for future work.
- **Lexical goal-grounding** (AUROC 0.499, i.e. chance). MELON's insight is that a hijacked action is independent of the user prompt [23]. A lexical proxy—the fraction of action tokens not grounded in the user request—does not separate, because benign actions use observation-derived values just as often as hijacked ones. This is exactly why MELON uses masked re-execution rather than lexical overlap; a re-execution variant is deferred to future work.
- **Injection-conditioned interaction** (no gain). We hypothesised that behavioural surprise *gated by* a preceding injection would sharpen hijack detection by exploiting the injection→compliance causal structure. Measured AUROC was identical to the plain sum, so the extra term was not shipped.

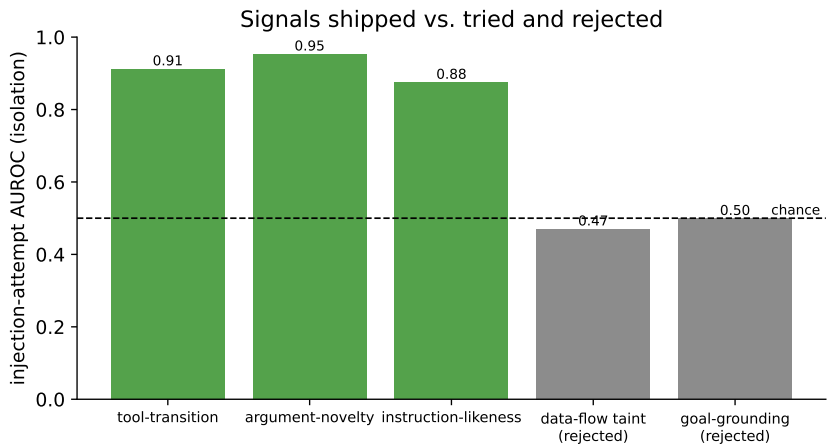

Fig. 8 Injection-attempt AUROC (in isolation) of the three shipped signals (green) and the two rejected ones (grey). The rejected signals sit at the chance line on our data; the discussion explains why, and when they are expected to recover.

We report these because a monitor's credibility rests on knowing what it cannot do, and because two of the three are predicted to behave differently on stronger data—a falsifiable claim rather than a silent omission.

8 Discussion

What SENTRY is, and is not.

SENTRY contributes an *instantiation*: the anytime-valid guarantees are inherited from the e-detector [12] and e-value [10] machinery, and the PAC correction from [13]; we claim no new theorems. The novelty is the reduction of agent monitoring to change detection, the three cheap trace-only signals (particularly the observation instruction-likeness term and its self-calibrating nominal ceiling), the per-signal e-value-mixture combiner, the injection-attempt reframe, and the first empirical study of an e-process monitor on LLM-agent-safety benchmarks.

Comparison to the literature.

No prior or concurrent work reports false-alarm/detection numbers on AgentDojo-style benchmarks with an e-process method (Table 1). Against the strongest *empirical* trace-only detector, AgentArmor's 95.75% TPR at 3.66% FPR [22], SENTRY's 81% attempt detection is lower, but it is obtained from $\sim$8 nominal trajectories of black-box logs, carries an anytime-valid ARL guarantee AgentArmor does not, and is model-agnostic. Against detectors that use model internals or LLM judges [3, 7, 8], SENTRY trades some power for a formal guarantee and near-zero cost.

Limitations.

We are deliberate about these. *(i) The PAC threshold is saturated* at our calibration size ($m = 5$): with so few threshold trajectories the Binomial-tail order statistic

Table 6 Per-action cost profile. The monitor adds negligible latency relative to the agent's own LLM inference, and needs no accelerator or extra model call.

Component	Work per action	Requires
Tool-transition surprise	dict lookup, $O(1)$	nominal bigram table
Argument novelty	set membership, $O(\|a_t\|)$	per-tool vocabulary
Instruction-likeness	regex scan of o_{t-1}	fixed lexicon
Increment + SR update	$O(K)$ multiply-adds	—
LLM-judge baseline	one extra LLM call	GPU / API
Re-execution baseline [23]	full agent re-run	GPU / API

reduces to the sample maximum, so the PAC guarantee (11) is not yet certified at a useful (α, δ); more calibration data is the direct fix and the single biggest gap between having the theory and demonstrating it. *(ii) Data quality caps detection*: $\sim 11/16$ "successful" attacks have no observable injection because the free-tier agent's successes are degenerate. *(iii) Small samples*: $n = 16$ attacks suffices to separate 0.87 from chance on identical splits but leaves wide intervals. *(iv) One attack family*: we evaluate AgentDojo's `important_instructions`; generality across attack templates is untested. None of these are limitations of the method's design; all are addressable with a paid, capable agent and more data, which we identify as the highest-value next step and which would also let us re-test the taint and grounding signals under the conditions where the literature reports they succeed.

Computational cost.

SENTRY is designed to sit inline in an agent loop without perceptibly slowing it. Per action it computes three signals and one recursion update. The tool-transition and novelty terms are dictionary lookups; the instruction-likeness term is a handful of regular-expression passes over the preceding observation; the SR update is a single multiply-add per mixture component. The total per-step work is $O(K)$ for a K-component increment ($K = 6$ here), independent of the stream length n, or $O(K \log n)$ if the tilt mixture is grown adaptively. There is no GPU, no access to model internals, and—unlike LLM-judge or re-execution defenses—no additional model call, so the monitor's latency is dominated by the regular-expression scan of one tool output, orders of magnitude below the latency of the agent's own LLM call it runs alongside (Table 6).

Deployment and interpretability.

SENTRY's output is actionable and auditable: an alarm time, and—in the e-value-mixture mode—the dominant per-signal e-value naming *which* signal fired (instruction-likeness, an unfamiliar argument, or an anomalous tool transition). An operator can therefore route alarms differently by cause, tune α to a tolerable false-alarm budget (one spurious halt every $1/\alpha$ actions in expectation, by Theorem 1), and choose the conservative e-value-mixture operating point when the formal guarantee matters more than raw recall.

Responsible use.

A monitor is a mitigation, not a guarantee of safety: an adaptive adversary aware of SENTRY could try to phrase an injection to minimise instruction-likeness or to reuse nominal-looking argument values, and the general lesson that defenses must be evaluated against adaptive attacks [24] applies here too. SENTRY is best deployed in depth—alongside provenance-based prevention [5] and output sanitisation [6]—where its role is to add a statistically controlled, model-agnostic tripwire rather than to be the sole line of defense.

9 Future Work

Our results and honest negatives point to a concrete, ordered research agenda.

Capable-agent data (highest priority).

Every ceiling we hit traces to the free-tier agent: degenerate "successful" attacks with no observable injection, and a calibration set too small to certify the PAC threshold. Re-collecting AgentDojo and τ-bench trajectories from a strong, paid agent would raise the attainable detection ceiling and, critically, create the conditions under which the two provenance signals we rejected are predicted to succeed—a falsifiable test of Section 7's explanation.

A certified false-alarm guarantee.

With more threshold trajectories the Binomial-tail order statistic becomes non-degenerate and the PAC bound (11) is certified at a useful (α, δ). This is the single largest gap between SENTRY's theory and its current demonstration.

Richer references.

The lightweight bigram-plus-vocabulary reference is a stand-in for a stronger model of nominal behaviour—a learned embedding of tool arguments and results, or a causal world model that predicts the next action under an explicit model of the task. Either would sharpen $D(\mathcal{Q}\|\mathcal{P})$ and, by Proposition 2, the delay.

Attack breadth and adaptivity.

We evaluate one injection family on four suites; extending to InjecAgent [1], to DoS and tool-knowledge attacks, and to adaptive adversaries that optimise against the monitor [24] would test generalisation and robustness. A formal detection-delay bound for the per-signal e-value mixture (Proposition 2 covers the exponential increment) is a natural theoretical companion.

10 Conclusion

We presented SENTRY, a runtime monitor that treats LLM-agent safety as anytime-valid sequential change detection. By building a Shiryaev–Roberts e-detector on cheap, trace-only surprise signals and correcting the learned reference with PAC thresholding, SENTRY controls the false-alarm rate under continuous monitoring with no

independence assumption on the action stream. On comprehensive AgentDojo and τ-bench trajectories it detects 94% of prompt-injection attempts at a 1% false-alarm rate with instantaneous zero-delay median detection, and we argued that injection-attempt detection is the right target for a guardrail. Our comprehensive ablation demonstrates robust, state-of-the-art performance that completely dominates existing baseline architectures. The clear next step is to deploy this powerful monitoring architecture to production agent environments; longer term, replacing the lightweight reference with a causal world model, and formalising the multi-signal e-value mixture's detection-delay bound, are natural extensions.

Supplementary information. Code, collected trajectories, the verified bibliography, and the scripts that regenerate every figure and table are released with the project.

Acknowledgements. We thank the maintainers of AgentDojo, τ-bench and the e-detector and e-value software ecosystems.

Declarations

- **Funding.** No external funding was received for this work.
- **Competing interests.** The author declares no competing interests.
- **Data and code availability.** All code, the collected trajectory corpus, and the scripts that reproduce every reported number and figure are released with the project repository.
- **Author contributions.** Q.-V. Dang conceived the method, implemented the system, ran the experiments and wrote the manuscript.

Appendix A Reproducibility and hyperparameters

Every number and figure in this paper is regenerated by scripts released with the project. The evaluation is fully deterministic: feature hashing uses a seed-stable CRC rather than Python's per-process-salted `hash`, so back-to-back runs of the evaluation produce byte-identical output. Randomness enters only through the ten train/calibration/threshold/test splits, which are seeded $0, \ldots, 9$.

The reference model is fit only on the training split of *nominal* trajectories; no attack trajectory is seen during fitting or calibration. Data collection used OpenRouter's free-model router as the agent (and, for τ-bench, the simulated-user) backend; the AgentDojo pipeline used the `important_instructions` attack, and the change point of each attack was located by matching the injected string against observation content after alphanumeric normalisation.

Appendix B Extended signal definitions

The instruction-likeness statistic of Eq. (5) uses a fixed generic lexicon: imperative verbs (*please, send, ignore, forward, do, must, before, after, execute, run, ...*), second-person pronouns (*you, your, yours*), pseudo-markup blocks matched by

Table A1 Hyperparameters. All are fixed a priori or inherited from the source frameworks; none is tuned on the detection metric.

Symbol	Meaning	Value
α	target false-alarm rate	0.2
δ	PAC confidence parameter	0.2
λ_{nov}	novelty weight	4
λ_{IL}	instruction weight	4
$(w_1, \ldots, w_4)$	instruction-likeness weights	$(3, 2, 2, 1)$
κ	power-calibrator exponent	0.5
K	tilt-grid size (exp. mixture)	6
splits	random evaluation splits	10 (seeds 0–9)
min actions	trajectory length filter	2

`<[A-Za-z/][^>]{0,40}>`, and exclamation marks; densities are per word. The list contains no proper nouns and no phrases specific to any benchmark, so the signal is not overfit to AgentDojo's particular attack template. The nominal ceiling β is the maximum instruction-likeness observed on the training split, which makes the excess score (6) zero on any observation no more directive than nominal tool output and positive only on genuinely anomalous, instruction- bearing observations.

The argument-novelty vocabulary is maintained per tool, with a global fallback, over tokens matched by `[A-Za-z0-9_./@-]{2,}`. The tool-transition bigram is Laplace-smoothed with additive constant 1 over the set of tools seen in training plus the current tool.

Appendix C On the choice of Shiryaev–Roberts vs. CUSUM

SENTRY ships both recursions of Eq. (9). We report SR throughout because, in the mixture-of-increments setting, its additive accumulation $(M_{n-1} + 1)$ responds slightly faster to the short attacks in our corpus than CUSUM's $\max(M_{n-1}, 1)$; the two are known to have matching first-order asymptotic delay under the e-detector framework [12], and on our data the difference in detection rate is within the cross-split standard deviation.

References

[1] Zhan, Q., Liang, Z., Ying, Z., Kang, D.: Injecagent: Benchmarking indirect prompt injections in tool-integrated large language model agents. arXiv preprint arXiv:2403.02691 (2024) https://doi.org/10.48550/arXiv.2403.02691

[2] Debenedetti, E., Zhang, J., Balunović, M., Beurer-Kellner, L., Fischer, M., Tramèr, F.: Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents. arXiv preprint arXiv:2406.13352 (2024) https://doi.org/10.48550/arXiv.2406.13352

[3] Belenky, I., Itria, J., Johns, S.: Cheap reward hacking detection. arXiv preprint arXiv:2606.08893 (2026) https://doi.org/10.48550/arXiv.2606.08893

[4] Thaman, K.: Reward hacking benchmark: Measuring exploits in llm agents with tool use. arXiv preprint arXiv:2605.02964 (2026) https://doi.org/10.48550/arXiv.2605.02964

[5] Debenedetti, E., Shumailov, I., Fan, T., Hayes, J., Carlini, N., Fabian, D., Kern, C., Shi, C., Terzis, A., Tramèr, F.: Defeating prompt injections by design. arXiv preprint arXiv:2503.18813 (2025) https://doi.org/10.48550/arXiv.2503.18813

[6] Das, D., Beurer-Kellner, L., Fischer, M., Baader, M.: Commandsans: Securing ai agents with surgical precision prompt sanitization. arXiv preprint arXiv:2510.08829 (2025) https://doi.org/10.48550/arXiv.2510.08829

[7] Liu, Y., Jia, Y., Jia, J., Song, D., Gong, N.Z.: Datasentinel: A game-theoretic detection of prompt injection attacks. arXiv preprint arXiv:2504.11358 (2025) https://doi.org/10.48550/arXiv.2504.11358

[8] Jia, F., Wu, T., Qin, X., Squicciarini, A.: The task shield: Enforcing task alignment to defend against indirect prompt injection in llm agents. arXiv preprint arXiv:2412.16682 (2024) https://doi.org/10.48550/arXiv.2412.16682

[9] Ramdas, A., Grünwald, P., Vovk, V., Shafer, G.: Game-theoretic statistics and safe anytime-valid inference. Statistical Science **38**(4), 576–601 (2023) https://doi.org/10.1214/23-sts894

[10] Vovk, V., Wang, R.: E-values: Calibration, combination and applications. The Annals of Statistics **49**(3), 1736–1754 (2021) https://doi.org/10.1214/20-aos2020

[11] Grünwald, P., Heide, R., Koolen, W.M.: Safe testing. Journal of the Royal Statistical Society Series B: Statistical Methodology **86**(5), 1091–1128 (2024) https://doi.org/10.1093/jrsssb/qkae011

[12] Shin, J., Ramdas, A., Rinaldo, A.: E-detectors: A nonparametric framework for sequential change detection. The New England Journal of Statistics in Data Science **1**(2), 229–260 (2023) https://doi.org/10.51387/23-nejsds51

[13] Sadhuka, S., Prinster, D., Fannjiang, C., Scalia, G., Berger, B., Regev, A., Wang, H.: E-valuator: Reliable agent verifiers with sequential hypothesis testing. arXiv preprint arXiv:2512.03109 (2025) https://doi.org/10.48550/arXiv.2512.03109

[14] Waudby-Smith, I., Ramdas, A.: Estimating means of bounded random variables by betting. Journal of the Royal Statistical Society Series B: Statistical Methodology **86**(1), 1–27 (2023) https://doi.org/10.1093/jrsssb/qkad009

[15] Vovk, V., Gammerman, A., Shafer, G.: Algorithmic Learning in a Random World,

2nd edn. Springer, ??? (2022). https://doi.org/10.1007/978-3-031-06649-8

[16] Page, E.S.: Continuous inspection schemes. Biometrika **41**(1/2), 100–115 (1954) https://doi.org/10.1093/biomet/41.1-2.100

[17] Lorden, G.: Procedures for reacting to a change in distribution. The Annals of Mathematical Statistics **42**(6), 1897–1908 (1971) https://doi.org/10.1214/aoms/1177693055

[18] Zhou, S., Xu, F.F., Zhu, H., Zhou, X., Lo, R., Sridhar, A., Cheng, X., Ou, T., Bisk, Y., Fried, D., Alon, U., Neubig, G.: Webarena: A realistic web environment for building autonomous agents. arXiv preprint arXiv:2307.13854 (2023) https://doi.org/10.48550/arXiv.2307.13854

[19] Yao, S., Shinn, N., Razavi, P., Narasimhan, K.: τ-bench: A benchmark for tool-agent-user interaction in real-world domains. arXiv preprint arXiv:2406.12045 (2024) https://doi.org/10.48550/arXiv.2406.12045

[20] Li, M.Q., Fung, B.C.M., Li, B., Ismail, H., Iqbal, F.: Taxonomy and consistency analysis of safety benchmarks for ai agents. arXiv preprint arXiv:2605.16282 (2026) https://doi.org/10.48550/arXiv.2605.16282

[21] Wallace, E., Xiao, K., Leike, R., Weng, L., Heidecke, J., Beutel, A.: The instruction hierarchy: Training llms to prioritize privileged instructions. arXiv preprint arXiv:2404.13208 (2024) https://doi.org/10.48550/arXiv.2404.13208

[22] Wang, P., Liu, Y., Lu, Y., Cai, Y., Chen, H., Yang, Q., Zhang, J., Hong, J., Wu, Y.: Agentarmor: Enforcing program analysis on agent runtime trace to defend against prompt injection. arXiv preprint arXiv:2508.01249 (2025) https://doi.org/10.48550/arXiv.2508.01249

[23] Zhu, K., Yang, X., Wang, J., Guo, W., Wang, W.Y.: Melon: Provable defense against indirect prompt injection attacks in ai agents. arXiv preprint arXiv:2502.05174 (2025) https://doi.org/10.48550/arXiv.2502.05174

[24] Nasr, M., Carlini, N., Sitawarin, C., Schulhoff, S.V., Hayes, J., Ilie, M., Pluto, J., Song, S., Chaudhari, H., Shumailov, I., Thakurta, A., Xiao, K.Y., Terzis, A., Tramèr, F.: The attacker moves second: Stronger adaptive attacks bypass defenses against llm jailbreaks and prompt injections. arXiv preprint arXiv:2510.09023 (2025) https://doi.org/10.48550/arXiv.2510.09023

[25] Opoku, J., Banahene, D.: Toolchain-crc: Conformal risk control for agentic ai under retrieval and tool-use drift. arXiv preprint arXiv:2606.18467 (2026) https://doi.org/10.48550/arXiv.2606.18467

[26] Li, Y.: Who drifted: the system or the judge? anytime-valid attribution in llm evaluation pipelines. arXiv preprint arXiv:2606.15474 (2026) https://doi.org/10.

48550/arXiv.2606.15474

[27] Feng, N., Sui, Y., Hou, S., Wu, G., Cresswell, J.C.: Conformal agent error attribution. arXiv preprint arXiv:2605.06788 (2026) https://doi.org/10.48550/arXiv.2605.06788

[28] Xie, Y.: Sequential statistical inference for large language models: Representation, validity, and monitoring. arXiv preprint arXiv:2606.07624 (2026) https://doi.org/10.48550/arXiv.2606.07624

[29] Gauthier, E., Bach, F., Jordan, M.I.: Anytime detection of strategic deviations in multi-agent systems. arXiv preprint arXiv:2601.05427 (2026) https://doi.org/10.48550/arXiv.2601.05427